

Laboratorio di Sistemi Operativi

Traccia A

PROF. ALBERTO FINZI



Universita' degli studi di Napoli Federico II

Dipartimento di Ingegneria Elettrica e dell'Informazione

Pasquale Junior Monto' N86005762
Pietro Pellegrino N86004722

22 maggio 2026

Indice

1	Introduzione	2
2	Guida d'uso	2
2.1	Compilazione	2
2.2	Avvio del server	2
2.3	Avvio del client	3
2.4	Uso interattivo	3
3	Protocollo applicativo	3
3.1	Struttura generale	3
3.2	Comandi client → server	3
3.3	Mappe locale e globale	4
3.4	Lista utenti e classifica	4
4	Dettagli implementativi	4
4.1	Architettura concorrente del server	4
4.2	Stati della sessione	4
4.3	Avvio della partita	5
4.4	Generazione del labirinto	5
4.5	Movimento, oggetti e uscita	5
4.6	Classifica e punteggio	6
4.7	Autenticazione e registrazione	6
4.8	Interfaccia utente (TUI)	6
4.9	Logging	7
4.10	Gestione delle disconnessioni	7
5	Mappa dei moduli	7
6	Scelte progettuali	7
7	Conclusioni	8

1 Introduzione

Il progetto implementa un sistema client-server concorrente in C per la gestione di una partita in un labirinto 2D con oggetti, uscite e più giocatori contemporanei. Il server mantiene la mappa completa, gestisce la posizione dei client, la scoperta progressiva delle celle e il punteggio finale. Il client fornisce un'interfaccia testuale interattiva (TUI) per login, registrazione, movimento, consultazione delle mappe, elenco utenti e classifica.

La soluzione segue la traccia A, rispettando l'uso di socket TCP, la separazione tra protocollo applicativo e livello di trasporto e la comunicazione asincrona dal server verso i client. Il codice è organizzato in moduli separati: `server/` (logica di gioco e gestione connessioni), `client/` (interfaccia utente e parsing risposte) e `common/` (protocollo e costanti condivise).

2 Guida d'uso

2.1 Compilazione

Il progetto usa un `Makefile` con i seguenti target principali:

```
make build # compila client e server (debug)
make run-server # compila e avvia il server sulla porta 8080
make run-client # compila e avvia il client su 127.0.0.1:8080
make test # compila ed esegue la suite di test automatici
```

La compilazione del server richiede il supporto ai thread POSIX (`-pthread`) e alla libreria `hiredis` per l'autenticazione via Redis. La compilazione del client non richiede dipendenze esterne oltre alla `libc` standard.

Il server è composto da `server/server.c`, `server/game.c`, `server/log.c`, `server/redis_threadsafe.c` e `common/protocol.c`. Il client da `client/client.c`, `client/ui.c`, `client/command.c`, `client/response.c`, `client/state.c` e `common/protocol.c`.

2.2 Avvio del server

Il server accetta opzionalmente una porta e un percorso per il file di log. I valori di default sono porta 8080 e log `logs/server.log`.

```
./server_app # default: porta 8080
./server_app 9090 # porta personalizzata
./server_app 8080 mylog.txt # porta e log personalizzati
```

All'avvio il server stampa a terminale:

```
=== LSO Server ===
Port: 8080
Log file: logs/server.log
Redis: connected to 127.0.0.1:6379
Server is running in LOBBY. Press Ctrl+C to stop.
```

Quindi:

- inizializza il logger e il socket TCP di ascolto;
- si connette a Redis su `127.0.0.1:6379` per gestire le credenziali utente (se Redis non è disponibile, l'autenticazione viene disabilitata e il server stampa un avviso);
- avvia il thread periodico per l'invio delle mappe globali mascherate (`periodic_global`). Il timer di sessione viene invece creato all'inizio di ogni partita, quando l'owner invia `START`.

2.3 Avvio del client

```
./client_app 127.0.0.1 8080
```

Il client usa `getaddrinfo()` per risolvere hostname e indirizzi numerici, connettendosi via socket TCP. Subito dopo la connessione, il terminale viene messo in modalità raw per consentire l'input immediato da tastiera (movimento diretto con i tasti W/A/S/D).

2.4 Uso interattivo

Il client prevede due modalità commutabili con il tasto TAB:

- **Command mode** – i comandi vengono digitati e confermati con Invio. Viene mostrato un prompt `>` con l'input buffer corrente e una lista dei comandi disponibili.
- **Movement mode** – i tasti W/A/S/D inviano immediatamente i movimenti al server. **G** richiede la mappa globale mascherata, **L** torna alla mappa locale. **Q** (maiuscola) invia **QUIT** e disconnette il client.

Comandi testuali supportati:

- **register <nickname> <password>** – registra un nuovo utente
- **login <nickname> <password>** – autentica un utente esistente
- **ready** – marca il giocatore come pronto nella lobby
- **start** – (solo owner) avvia la partita
- **reset** – (solo owner) riporta la sessione in lobby dopo la fine
- **rank** – richiede la classifica
- **users** – richiede la lista degli utenti connessi
- **local** – richiede la mappa locale (finestra 5×5)
- **global** – richiede la mappa globale mascherata
- **quit** – disconnette il client

3 Protocollo applicativo

3.1 Struttura generale

Il protocollo è line-based: ogni messaggio è una riga testuale terminata da `\n`. I messaggi multi-linea (mappe, liste utenti, classifica) terminano con una riga **END**. Prefissi testuali distinguono i tipi di risposta (**OK**, **ERR**, **SESSION**, **MAP**, **USERS**, **RANK**, **TIME**).

3.2 Comandi client → server

I comandi vengono tradotti dal modulo `client/command.c` nel formato protocollare. Il client supporta anche varianti con slash iniziale (`/ready` $\equiv$ `ready`) e gestisce il comando **help** localmente senza inviare dati al server.

Prefisso	Esempio	Descrizione
OK	OK authenticated	Operazione riuscita
ERR	ERR wrong password	Errore
SESSION	SESSION STARTED	Evento di sessione
MAP LOCAL	MAP LOCAL 5 5	Mappa locale (finestra 5×5)
MAP GLOBAL	MAP GLOBAL 21 21	Mappa globale mascherata
USERS	USERS 3	Lista utenti connessi
RANK	RANK 3	Classifica finale
TIME	TIME 245	Tempo rimanente in secondi

Tabella 1: Prefissi del protocollo di risposta

3.3 Mappe locale e globale

La mappa locale (5×5 celle, centrata sul giocatore) mostra solo le celle già scoperte dal giocatore. La mappa globale (21×21 celle) rappresenta l'intera matrice di gioco con le celle non ancora esplorate sostituite dal simbolo ? (cella nascosta). Il server invia la mappa globale periodicamente ogni `T_INTERVAL` (5 secondi) a tutti i client durante la partita.

3.4 Lista utenti e classifica

La lista utenti è inviata automaticamente dopo login, register, ready, reset e disconnessione. Ogni riga mostra il nickname con marcatori `[owner]` e `[ready]`. La classifica combina i punteggi dei giocatori ancora connessi e di quelli già disconnessi, ordinati per punteggio decrescente.

4 Dettagli implementativi

4.1 Architettura concorrente del server

Il server è multithread:

- un thread per ogni client connesso (creato con `pthread_create` e detached con `pthread_detach`);
- un thread periodico (`periodic_global`) che ogni `T_INTERVAL` secondi invia la mappa globale mascherata e il tempo rimanente a tutti i client;
- un thread `session_timer` che decreta la fine della sessione dopo `SESSION_DURATION` (300) secondi.

La sincronizzazione è realizzata con cinque mutex indipendenti:

- `maze_mutex` – protegge la matrice del labirinto;
- `client_mutex` – protegge la lista dei client connessi;
- `score_mutex` – protegge la tabella dei punteggi;
- `session_mutex` – protegge lo stato della sessione e il timer;
- `redis_mutex` (nel modulo `redis_threadsafe`) – serializza l'accesso al contesto Redis condiviso, evitando accessi concorrenti da parte dei thread client durante login e registrazione.

La separazione dei mutex riduce la contesa tra i thread.

4.2 Stati della sessione

Il server gestisce tre stati:

- **LOBBY** – i client possono registrarsi, autenticarsi e dichiararsi pronti (**READY**). Solo l'owner può avviare la partita con **START**.
- **PLAYING** – la partita è attiva. I client possono muoversi, raccogliere oggetti, consultare le mappe. Il server invia periodicamente la mappa globale e il tempo rimanente.
- **FINISHED** – la partita è terminata. I client possono consultare la classifica (**RANK**). Solo l'owner può resettare la sessione (**RESET**) per tornare in **LOBBY**.

4.3 Avvio della partita

La partita parte quando l'owner invia **START** e *tutti* i giocatori non-owner connessi e autenticati hanno inviato **READY**. Se qualche giocatore non è pronto, il server risponde con **ERR not all players are ready**.

All'avvio:

1. il labirinto viene generato (algoritmo ricorsivo di backtracking, matrice 21×21 , percorsi larghi una cella tra pareti spesse);
2. vengono piazzati 15 oggetti in celle vuote casuali e un'uscita su uno dei quattro bordi;
3. a ogni client viene assegnata una posizione di spawn casuale (round-robin da un pool di celle walkable a coordinate dispari);
4. viene avviato il timer di 300 secondi.

4.4 Generazione del labirinto

Il labirinto è generato con backtracking ricorsivo (depth-first search):

1. l'intera matrice viene riempita di muri (#);
2. partendo dalla cella (1,1), si sceglie una direzione casuale con passo 2 e si apre la parete intermedia;
3. si ricorre sulla cella di destinazione;
4. la randomizzazione delle direzioni a ogni passo garantisce labirinti diversi a ogni esecuzione.

4.5 Movimento, oggetti e uscita

Il movimento è a singola cella nelle quattro direzioni cardinali. Il server controlla che la cella di destinazione sia raggiungibile (non muro). In caso di movimento valido:

- l'area di raggio **VIEW_RADIUS** (2) attorno al giocatore viene marcata come visibile in modo permanente;
- se la cella contiene un oggetto, questo viene raccolto e rimosso dalla mappa;
- se la cella è l'uscita, il giocatore viene marcato come **exit_reached** (il suo marcatore scompare per gli altri giocatori e lui passa in modalità spettatore potendo osservare la mappa globale).

La sessione termina quando *tutti* i giocatori hanno raggiunto l'uscita, oppure allo scadere del timer (300 secondi), oppure se tutti i giocatori si disconnettono.

4.6 Classifica e punteggio

Il punteggio è calcolato con la formula:

$$\text{score} = \text{objects_collected} \times 100 - \text{exit_time_seconds}$$

dove `exit_time_seconds` è il tempo trascorso dall'inizio della sessione al momento in cui il giocatore ha raggiunto l'uscita. I giocatori che non raggiungono l'uscita non ricevono punteggio e sono ordinati per numero di oggetti raccolti.

L'ordinamento usa `qsort` con comparatore personalizzato: prima i giocatori usciti (per punteggio decrescente), poi i non usciti (per numero di oggetti decrescente).

La risposta `RANK` include per ogni giocatore il numero di oggetti raccolti, il tempo di uscita (se applicabile) e il punteggio calcolato.

4.7 Autenticazione e registrazione

Il server usa Redis come archivio esterno per le credenziali. La chiave usata è `user:<nickname>` con valore la password. Le password sono memorizzate in chiaro: questa scelta è stata adottata per semplicità didattica e per concentrare il progetto sugli aspetti di comunicazione, concorrenza e gestione della sessione. In un sistema reale sarebbe necessario usare hashing sicuro delle password.

- **Registrazione:** controlla che il nickname non sia già presente in Redis e che non inizi con `guest` (riservato ai client non autenticati). In caso positivo, memorizza la coppia nickname-password.
- **Login:** recupera la password da Redis e la confronta con quella fornita. Se il nickname non esiste risponde `ERR user not registered`; se la password non corrisponde risponde `ERR wrong password`.
- **Occupazione nickname:** controlla che nessun altro client connesso stia già usando quel nickname, per evitare conflitti.

Se Redis non è disponibile all'avvio, il server stampa un avviso e le operazioni di autenticazione rispondono con un errore.

4.8 Interfaccia utente (TUI)

Il client disegna un'interfaccia testuale persistente usando sequenze di escape ANSI e caratteri box-drawing UTF-8 (bordi orizzontali e verticali), evitando dipendenze da ncurses. La schermata viene ridisegnata a ogni cambiamento di stato.

La TUI mostra:

- in **LOBBY**: il titolo, le istruzioni per owner/non-owner, la lista dei giocatori connessi con colori distinti (owner in verde grassetto, giocatori pronti in giallo, guest in blu);
- in **PLAYING**: la mappa locale o globale con legenda colorata (P=giocatore in verde, 0=altri in giallo, *=oggetti in ciano, E=uscita in magenta, ?=nascosto in blu, #=muro), il timer e i controlli disponibili;
- in **FINISHED**: la classifica con colori per le prime tre posizioni (oro, argento, bronzo) e il punteggio in magenta.

La funzione `clear_screen()` invia `\033[2J\033[H` per pulire lo schermo a ogni frame. La funzione `visible_width()` gestisce correttamente le sequenze multi-byte UTF-8 e i codici di escape ANSI per mantenere l'allineamento dei bordi.

4.9 Logging

Il server registra in `logs/server.log` tutti gli eventi principali: avvio, connessioni (`CONNECTED: <nick>`), disconnessioni (`DISCONNECTED: <nick>`), inizio/fine/reset sessione. Ogni riga è preceduta da un timestamp nel formato `[YYYY-MM-DD HH:MM:SS]`. La scrittura è protetta da mutex poiché più thread client possono scrivere contemporaneamente. I messaggi di log vengono anche stampati su `stderr` per facilitare il debug in tempo reale.

4.10 Gestione delle disconnessioni

Quando un client si disconnette durante la partita:

- il suo punteggio viene salvato nella tabella `known_scores` per apparire nella classifica finale;
- se era l'owner, la proprietà passa al primo client rimanente;
- se non rimangono client connessi, la sessione passa a `FINISHED`;
- se tutti i giocatori rimanenti hanno già raggiunto l'uscita, la sessione termina immediatamente.

5 Mappa dei moduli

Modulo	File	Responsabilità
Server core	<code>server/server.c</code>	Main loop, accept, gestione comandi, stati
Game logic	<code>server/game.c</code>	Labirinto, movimento, visibilità, mappe
Logging	<code>server/log.c</code>	Scrittura log con timestamp e mutex
Redis thread-safe	<code>server/redis_threadsafe.c</code>	Mutex per contesto Redis condiviso
Client core	<code>client/client.c</code>	Connessione, select loop, dispatch input
TUI	<code>client/ui.c</code>	Rendering schermo, bordi, mappe, colori
Commands	<code>client/command.c</code>	Traduzione comandi utente → protocollo
Response parser	<code>client/response.c</code>	Parsing risposte server multi-linea
Client state	<code>client/state.c</code>	Stato client, autenticazione pendente
Protocollo	<code>common/protocol.h</code>	Costanti comandi/risposte, <code>send_all</code> , <code>recv_line</code>
Costanti	<code>common/constants.h</code>	Dimensioni, simboli, colori ANSI

Tabella 2: Organizzazione modulare del progetto

6 Scelte progettuali

Le principali scelte progettuali sono state:

1. **Separazione client/server/protocollo:** ogni livello è indipendente e testabile singolarmente. Il client non conosce la logica di gioco, il server non conosce l'interfaccia utente.

2. **Protocollo testuale line-based:** semplifica il debugging (i messaggi sono leggibili con `telnet`) e il parsing (ogni riga è autoconclusiva fino a `\n`).
3. **Server multithread con mutex granulari:** un thread per client più thread ausiliari, con mutex separati per mappa, lista client, punteggi, stato sessione e contesto Redis.
4. **TUI senza ncurses:** l'interfaccia usa solo sequenze ANSI e caratteri UTF-8, senza dipendenze esterne, funzionante su qualsiasi terminale moderno.
5. **Redis per credenziali:** archivio esterno leggero che persiste le registrazioni finché il server Redis è attivo. In caso di indisponibilità il server funziona comunque (con autenticazione disabilitata).
6. **Test automatici:** suite di test che coprono il parser comandi, lo stato client, il parser risposte, il protocollo e la logica di gioco (movimento, visibilità, pickup, uscita).

7 Conclusioni

Il progetto soddisfa i requisiti della traccia A: gestione concorrente di più client, labirinto con oggetti e uscita, scoperta progressiva delle celle, invio periodico della mappa globale mascherata, autenticazione utenti, logging e classifica finale con punteggio composito (oggetti $\times$ tempo di uscita). La struttura modulare, la suite di test e l'uso di strumenti standard (Makefile, Redis, ANSI escapes) rendono il sistema manutenibile e portabile.

Tutto il codice sorgente, i test e la documentazione sono disponibili nel repository del progetto.